

Atividade: Explorando o Código de Inicialização e o *Linker Script*

Na aula, você elaborou o `startup.c` e o `stm32f411-rom.ld`, e verificou que a imagem do `embedded.elf` ficou posicionada exatamente onde o processador irá procurá-la. Nesta atividade, você manipulará essas duas peças, uma de cada vez, e observará o efeito de cada modificação: completar a tabela de vetores, substituir um tratador, verificar onde o compilador posiciona cada tipo de variável, examinar o código de máquina do tratador de reset com e sem otimização, provocar deliberadamente falhas no *linker script* e reservar um setor da Flash.

Todo o trabalho ocorre no PC, com as ferramentas já utilizadas anteriormente: `arm-none-eabi-gcc`, `nm`, `objdump`, `od` e o arquivo `.map`. Não há, portanto, necessidade da placa.

i Orientações

- **Pré-requisito:** a prática da aula concluída, com a pasta `~/seml-workspace/lab-03` contendo o `embedded.c`, o `startup.c` e o `stm32f411-rom.ld`, ligando sem qualquer erro. Todos os comandos desta atividade devem ser executados nessa mesma pasta.
- **Não estrague o projeto da aula.** Cada experimento deve criar arquivos **novos**, com outro nome (por exemplo, `startup-o2.o` ou `sem-keep.ld`). O `startup.c` e o `stm32f411-rom.ld` originais sofrem alteração apenas na Parte 1, e a versão final de ambos constitui parte da entrega.
- **Previsões:** diversos experimentos solicitam uma previsão **antes** da execução do comando correspondente. Registre-a exatamente como formulada, sem corrigi-la posteriormente. Errar a previsão constitui parte legítima do exercício; o que efetivamente se avalia é a qualidade da explicação do resultado obtido.
- **Saídas esperadas:** os experimentos apresentam as saídas obtidas com o `arm-none-eabi-gcc` versão 15. Com uma versão distinta, endereços e tamanhos podem apresentar pequenas variações. Para obter as mensagens do compilador e do *linker* em inglês, como reproduzido aqui, inicie o comando com `LANG=C`.
- **Entrega:** a pasta `lab-03`, em seu repositório Git privado (configurado no roteiro de Ambiente e Ferramentas de Desenvolvimento), contendo o projeto final da Parte 1 e um arquivo `LEIAME.md` com as respostas da seção Relatório, incluindo as saídas dos comandos que as justificam.
- **Respostas autorais:** elabore explicações próprias, fundamentadas nos experimentos realizados. Respostas copiadas de fontes externas, ou geradas por IA sem elaboração própria, não atingem o objetivo pedagógico da atividade.

Antes de iniciar, acesse a pasta e verifique que o projeto elaborado na aula ainda se liga corretamente:

```
foo@bar$ cd ~/seml-workspace/lab-03
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -
```

```
o startup.o
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall embedded.c -
o embedded.o
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld -Wl,--print-memory-
usage startup.o embedded.o -o embedded.elf
Memory region      Used Size  Region Size  %age Used
      FLASH:        276 B      512 KB      0.05%
      SRAM:         12 B      128 KB      0.01%
```

Esses três comandos se repetem ao longo de toda a atividade. Sempre que o texto indicar “ligue de novo”, é a eles que a instrução se refere, com a devida substituição dos nomes de arquivo quando especificado.

Parte 1 — A tabela de vetores completa

A tabela elaborada na aula contém apenas as 16 posições definidas pela arquitetura. A tabela completa do STM32F411, entretanto, prossegue com uma posição destinada a cada interrupção de periférico — as IRQ 0 a 85 —, descritas na *Table 37* do RM0383 (*Vector table for STM32F411xC/E*). Cada linha da tabela apresentada no manual traz a posição, a prioridade, o nome, a descrição e o endereço correspondentes.

Antes de alterar o `startup.c`, registre o tamanho atual da `.text`, obtido por meio de `arm-none-eabi-objdump -h embedded.elf`. **Previsão:** quantos bytes a `.text` deverá crescer quando a tabela estiver completa?

Em seguida, complete a tabela no `startup.c`, em dois pontos distintos do arquivo:

1. Os protótipos. Imediatamente após a última linha de protótipo (a do `systick_handler`), **acrescente** um protótipo `weak` com `alias` para cada interrupção. O nome deve seguir o acrônimo indicado na coluna *Acronym* da tabela, em letras minúsculas e terminado em `_irq_handler`. Quando o acrônimo apresentar dois nomes separados por uma barra, utilize o segundo. As três primeiras entradas ficam assim:

```
void wwdg_irq_handler(void)
    __attribute__((weak, alias("default_handler")));           /* IRQ 0 */
void pvd_irq_handler(void)
    __attribute__((weak, alias("default_handler")));           /* IRQ 1: EXTI16 /
PVD */
void tamper_irq_handler(void)
    __attribute__((weak, alias("default_handler")));           /* IRQ 2: EXTI21 /
TAMP_STAMP */
```

2. A tabela. No interior de `g_vectors`, imediatamente após a linha correspondente ao `systick_handler` e antes do `};`, **acrescente** uma linha para cada posição, de 0 a 85, respeitando a ordem, acompanhada de um comentário que indique o número da IRQ e o endereço constante da coluna *Address*:

```
(uint32_t)wdg_irq_handler,      /* 0x0000 0040: IRQ 0 */
(uint32_t)pvd_irq_handler,      /* 0x0000 0044: IRQ 1 */
(uint32_t)tamp_stamp_irq_handler, /* 0x0000 0048: IRQ 2 */
```

Atenção às lacunas existentes. A tabela do manual **não lista** as posições reservadas: após a posição 51 (SPI3), por exemplo, a linha seguinte corresponde diretamente à posição 56 (DMA2_Stream0). Cada posição ausente na tabela deve ocupar uma palavra com o valor 0, acompanhada de um comentário *reservado*. Na ausência dessas palavras, todas as interrupções subsequentes ficariam deslocadas de sua posição correta. Para verificar, compare o endereço indicado no comentário de cada linha com a posição efetivamente ocupada por ela no vetor: a IRQ n deve situar-se em $0x40 + 4 \times n$.

Ligue novamente e verifique o novo tamanho da `.text`. Verifique também, por meio do `od`, as últimas palavras da tabela:

```
foo@bar$ arm-none-eabi-nm -n embedded.elf | head -3
foo@bar$ arm-none-eabi-objcopy -O binary embedded.elf embedded.bin
foo@bar$ od -A x -t x4 -j 0x188 -N 16 embedded.bin
```

O `od` exibe as quatro últimas palavras da tabela, que deve terminar precisamente no endereço `0x08000198` (408 bytes a partir de `0x08000000`). O `reset_handler` deverá ter mudado de endereço. Esta versão do `startup.c` é a que deve constar na entrega final.

Parte 2 — Um tratador de verdade

Os atributos `weak` e `alias` prometem que qualquer arquivo do projeto é capaz de substituir um tratador sem qualquer necessidade de alteração em `startup.c`. Vejamos essa afirmação na prática. Copie o `embedded.c` para `embedded-systick.c` e acrescente, ao final da cópia, uma função com o mesmo nome do tratador do SysTick:

```
void systick_handler(void)
{
    g_c++;
}
```

Previsão: a palavra da tabela de vetores correspondente ao SysTick (a última das 16 posições definidas pela arquitetura, no deslocamento `0x3c`) permanecerá apontando para `default_handler`? Ligue o projeto substituindo o `embedded.c` pelo `embedded-systick.c`, gerando `systick.elf` e `systick.bin`, e verifique:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall embedded-
systick.c -o embedded-systick.o
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld startup.o embedded-
systick.o -o systick.elf
```

```
foo@bar$ arm-none-eabi-objcopy -O binary systick.elf systick.bin
foo@bar$ arm-none-eabi-nm -n systick.elf | grep handler
foo@bar$ od -A x -t x4 -j 0x30 -N 16 systick.bin
```

Com o `startup.c` elaborado na aula (anterior à Parte 1), a saída do `nm` exibe `systick_handler` identificado pela letra `T`, ocupando um endereço próprio, ao passo que os demais tratadores permanecem com a letra `W`, no endereço de `default_handler`:

```
080000c4 T default_handler
...
08000110 T systick_handler
```

O `od`, por sua vez, mostra a posição correspondente ao SysTick apontando para a nova função, com o bit Thumb devidamente ativado:

```
000030 080000c5 00000000 080000c5 08000111
```

Com o `startup.c` produzido na Parte 1, os endereços obtidos serão distintos, mas a conclusão permanece idêntica.

Parte 3 — Onde fica cada variável

Na aula teórica, verificou-se que as partes de um programa se distribuem entre a Flash e a SRAM conforme sejam, ou não, alteradas durante a execução. Do ponto de vista da linguagem C, é a própria declaração da variável que determina a seção à qual ela pertence:

- uma variável **global** ou declarada com **static** existe durante toda a execução do programa e possui endereço fixo. Ela é destinada à `.data`, caso possua um valor inicial diferente de zero, ou à `.bss`, caso contrário;
- uma variável **const** global não sofre alteração, razão pela qual o compilador a posiciona na `.rodata`;
- um **literal de string**, como `"ola"`, também é destinado à `.rodata`;
- uma variável **local** comum existe somente enquanto a função correspondente está em execução, residindo na **pilha**. Ela não possui, portanto, seção nem símbolo associados.

Crie o arquivo `variaveis.c`, uma versão do `embedded.c` contendo uma variável de cada tipo mencionado:

```
int g_a = 10;
int g_b;
int g_c;
const int g_d = 20;
const char *g_msg = "ola";
```

```

int add(int x, int y)
{
    static int calls;
    calls++;
    return x + y;
}

int main(void)
{
    int local = 5;
    g_c = add(g_a, g_b) + local + g_d;
    while (1)
    {
    }
    return 0;
}

```

Previsão: antes de proceder à compilação, preencha uma tabela contendo uma linha para cada um dos seguintes itens: `g_d`, `g_msg`, o texto "ola", `calls` e `local`. Para cada um deles, indique a seção correspondente (`.data`, `.bss`, `.rodata` ou pilha) e a memória correspondente (Flash ou SRAM). Em seguida, compile, ligue e verifique:

```

foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall variaveis.c
-o variaveis.o
foo@bar$ arm-none-eabi-nm variaveis.o
foo@bar$ arm-none-eabi-objdump -h variaveis.o
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld -Wl,--print-memory-
usage startup.o variaveis.o -o variaveis.elf
foo@bar$ arm-none-eabi-nm -n variaveis.elf

```

Pontos que merecem observação e explicação no relatório:

- no `variaveis.o`, o símbolo `g_d` aparece identificado pela letra **R** (*read-only*), mas no `variaveis.elf` passa a exibir a letra **T**. A que se deve essa mudança? (Pista: observe o que o *linker script* realiza com as seções `.rodata*`.)
- `g_msg` e o texto "ola" encontram-se em locais distintos. Onde está localizado cada um deles, e por qual razão? O que precisaria ser modificado na declaração para que o próprio ponteiro também fosse destinado à Flash?
- o símbolo `calls` aparece com o nome `calls.0` e identificado pela letra **minúscula b**. O que essa letra minúscula indica? Por que motivo o compilador acrescentou o sufixo `.0` ao nome?
- a variável `local` não figura em nenhuma das listagens. Onde ela se encontra durante a execução do programa?
- a utilização de SRAM passou de 12 para 20 bytes. Quais variáveis justificam esses 8 bytes adicionais?

Parte 4 — O tratador de reset em código de máquina

Na prática de Do C ao Binário, procedeu-se à leitura da desmontagem do `main` do `embedded.c`. Chegou, agora, a vez do `reset_handler`, examinado em três níveis distintos de otimização. Gere os objetos correspondentes, com nomes diferentes:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall startup.c -o startup-o0.o
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O1 -Wall startup.c -o startup-o1.o
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O2 -Wall startup.c -o startup-o2.o
foo@bar$ arm-none-eabi-objdump -d startup-o0.o | less
```

Utilize os recursos de busca do `less` (`/reset_handler`) para localizar a função. Repita o procedimento para os demais dois objetos.

Com -O0, localize os dois laços presentes (a cópia da `.data` e a zeragem da `.bss`), as instruções responsáveis por cada comparação e cada salto de retorno, além da chamada a `main`. Observe quantas vezes as variáveis `i`, `dst` e `src` são lidas e gravadas na pilha (endereços relativos a `r7`).

Com -O1, o laço de cópia reduz-se a apenas quatro instruções:

```
10: f813 1b01 ldrb.w r1, [r3], #1
14: f802 1b01 strb.w r1, [r2], #1
18: 4298      cmp r0, r3
1a: d1f9      bne.n 10 <reset_handler+0xe>
```

A instrução `ldrb.w r1, [r3], #1` corresponde ao modo de endereçamento **pós-indexado**, discutido na Questão 5 do Estudo Dirigido: Arquitetura ARM Cortex-M: ela realiza a leitura de um byte no endereço apontado por `r3` e, **somente depois**, soma 1 a esse mesmo registrador. Trata-se da tradução direta da expressão `*dst++ = *src++`.

Com -O2, os laços desaparecem por completo. **Previsão:** o que você supõe que o compilador tenha posicionado em seu lugar? Verifique na desmontagem e, posteriormente, tente ligar esse objeto:

```
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld startup-o2.o embedded.o -o o2.elf
```

O *linker* recusa-se a efetuar a ligação, produzindo mensagens semelhantes a estas (o caminho do arquivo foi abreviado):

```
ld: startup-o2.o: in function `reset_handler':
startup.c:70:(.text+0x10): undefined reference to `memcpy'
```

```
...
startup.c:80:(.text+0x1e): undefined reference to `memset'
collect2: error: ld returned 1 exit status
```

O compilador identificou que os dois laços realizam exatamente aquilo que as funções `memcpy` e `memset` da biblioteca C realizam, e substituiu os laços por chamadas diretas a elas. Ocorre que, sob a opção `-nostdlib`, essas funções simplesmente não existem. O manual do GCC esclarece que, mesmo em um programa desprovido de biblioteca C, o compilador pode gerar chamadas a `memcpy`, `memmove`, `memset` e `memcmp`, cabendo ao programador a responsabilidade de fornecê-las. Para evitar essa substituição, compile com a opção `-fno-tree-loop-distribute-patterns` e examine a desmontagem novamente.

Parte 5 — Quebrando o *linker script* e o *startup* de propósito

Cada experimento desta parte consiste em criar uma versão deliberadamente incorreta de um arquivo, sob outro nome, e observar o que ocorre. Nenhum deles deve, em qualquer circunstância, ser gravado na placa.

A .data sem AT>. Copie o `stm32f411-rom.ld` para `sem-at.ld` e, na cópia, substitua `> SRAM AT> FLASH` por apenas `> SRAM`, na seção `.data`. **Previsão:** o que se altera no LMA da `.data`? Ligue novamente, com `-T sem-at.ld`, gerando `sem-at.elf`, e verifique por meio de `objdump -h`. Em seguida, gere o `.bin` correspondente e observe o seu tamanho:

```
foo@bar$ arm-none-eabi-objcopy -O binary sem-at.elf sem-at.bin
foo@bar$ ls -l sem-at.bin
```

O arquivo resultante possui cerca de 384 MB. **Apague-o** imediatamente após registrar o seu tamanho (`rm sem-at.bin`).

O símbolo sem o &. Copie o `startup.c` para `sem-e-comercial.c` e, na cópia, substitua `dst = (uint8_t *)&_sdata;` por `dst = (uint8_t *)_sdata;`. **Previsão:** o compilador emitirá alguma reclamação? Compile a cópia com `-O0` e compare a desmontagem do início do `reset_handler` com aquela de `startup-o0.o`, obtida na Parte 4. Na versão incorreta, surge uma instrução adicional imediatamente após o endereço de `_sdata` ser carregado a partir da *literal pool*:

```
e: 4b1a      ldr r3, [pc, #104]
10: 681b      ldr r3, [r3, #0]
```

O que, exatamente, realiza essa segunda instrução? Qual valor `dst` recebe na versão incorreta, e por que essa situação é particularmente grave imediatamente após o reset?

A tabela de vetores sem KEEP. Em projetos de maior porte, é comum solicitar ao *linker* que descarte as seções não utilizadas por nenhuma função, com o propósito de economizar Flash. Para tanto, compila-se com `-ffunction-sections -fdata-sections` (uma seção distinta para cada

função e cada variável) e liga-se com `-Wl,--gc-sections` (*garbage collection* de seções). Gere os objetos correspondentes, sob outros nomes:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall -ffunction-  
sections -fdata-sections startup.c -o startup-gc.o  
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall -ffunction-  
sections -fdata-sections embedded.c -o embedded-gc.o  
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld -Wl,--gc-sections -  
Wl,--print-memory-usage startup-gc.o embedded-gc.o -o gc.elf
```

Copie, agora, o `stm32f411-rom.ld` para `sem-keep.ld`, substitua `KEEP*(.isr_vectors))` por apenas `*(.isr_vectors)`, e ligue os mesmos objetos com `-T sem-keep.ld`, gerando `sem-keep.elf` e `sem-keep.bin`. **Previsão:** o que se altera? Verifique com `nm -n` e com `od -A x -t x4 -N 8` sobre o `.bin`. Compare a utilização de Flash entre as duas ligações e explique o que o processador leria como topo da pilha e como endereço do tratador de reset, caso essa imagem viesse a ser efetivamente gravada. Por qual razão o `reset_handler` sobreviveu ao processo de coleta, mesmo sem que nenhuma função o chamasse?

Parte 6 — Reservando um setor da Flash

O bloco `MEMORY` constitui também o local apropriado para reservar memória. Suponha uma aplicação que armazena configurações na Flash e decide utilizar, para esse fim, o último setor — o setor 7, de `0x08060000` a `0x0807FFFF`: basta reduzir a região `FLASH` até excluí-lo. Copie o `stm32f411-rom.ld` para `reserva.ld` e, na cópia, reduza o `LENGTH` da região `FLASH` de modo a excluir o setor 7. Ligue novamente com `-T reserva.ld` e verifique, por meio do `--print-memory-usage`, o novo tamanho da região.

Faça, agora, com que o programa não caiba na região disponível. Crie o arquivo `tabela.c` contendo uma tabela constante de 400 KB:

```
const unsigned char g_table[400 * 1024] = { 1 };
```

Compile-o e ligue-o em conjunto com o programa, primeiramente utilizando o *linker script* original e, em seguida, o `reserva.ld`:

```
foo@bar$ arm-none-eabi-gcc -c -g -mcpu=cortex-m4 -mthumb -O0 -Wall tabela.c -  
o tabela.o  
foo@bar$ arm-none-eabi-gcc -nostdlib -T stm32f411-rom.ld -Wl,--print-memory-  
usage startup.o embedded.o tabela.o -o tabela.elf  
foo@bar$ arm-none-eabi-gcc -nostdlib -T reserva.ld -Wl,--print-memory-usage  
startup.o embedded.o tabela.o -o tabela-reserva.elf
```

Com o *linker script* original, o programa cabe integralmente, utilizando cerca de 78% da Flash. Com o `reserva.ld`, contudo, o *linker* recusa a ligação:

```
ld: tabela-reserva.elf section `.text' will not fit in region `FLASH'
ld: region `FLASH' overflowed by 16660 bytes
```

Esses números correspondem ao `startup.c` elaborado na aula; com a tabela de vetores completa, produzida na Parte 1, o excesso torna-se um pouco maior. Observe que nenhuma parte do programa efetivamente utiliza `g_table`, e, ainda assim, ela ocupa espaço na Flash. Com base no que foi observado na Parte 5, proponha e teste uma forma de fazer com que o *linker* a descarte.

Parte 7 — Lendo o `embedded.map` e a diferença entre variável e símbolo

A opção `-Wl,-Map=embedded.map`, empregada na prática, gerou um relatório completo do processo de ligação. Abra-o no VS Code:

```
foo@bar$ code embedded.map
```

Localize duas seções específicas do arquivo:

- **Memory Configuration**, situada logo no início: compare-a com o bloco `MEMORY` do `stm32f411-rom.ld` e com a saída de `--print-memory-usage`.
- **Linker script and memory map**, consideravelmente mais extensa: para cada seção de saída (`.text`, `.data`, `.bss`), lista as seções de entrada de cada arquivo objeto que ali foram encaixadas, na ordem em que efetivamente aparecem no executável. Localize a linha correspondente a `startup.o` que contribuiu com `.isr_vectors`, bem como a de `embedded.o` que contribuiu com `.text`.

Retome, agora, a distinção entre **variável** e **símbolo**, apresentada na aula (Parte 1 do `startup.c`, Passo 6). Na Parte 5, você já observou o efeito prático de omitir o `&` antes de `_sdata`. Para concluir o raciocínio, responda às questões a seguir, sem executar qualquer comando adicional:

- Por que motivo `int g_a = 10;` reserva memória e, simultaneamente, cria um símbolo, ao passo que `_sdata = .;`, no *linker script*, cria exclusivamente o símbolo?
- Caso `_sdata` fosse declarada como `extern uint32_t _sdata[10];`, em vez de `extern uint32_t _sdata;`, o programa continuaria funcionando corretamente? O que se alteraria na expressão `&_sdata`?
- No `embedded.map`, localize a linha correspondente ao símbolo `_sdata`. Ela figura na seção `.data`, juntamente com as variáveis de `embedded.o`, ou em uma lista separada? O que essa constatação confirma acerca de onde o símbolo efetivamente “mora”?

Relatório

Escreva as respostas no `LEIAME.md`, com suas próprias palavras, apresentando as saídas dos comandos que justificam cada uma delas:

1. **(Parte 1)** Qual foi a previsão formulada para o crescimento da `.text`, e qual foi o crescimento efetivamente observado? Explique a diferença, relacionando-a ao número de posições da tabela. Por que razão o endereço do `reset_handler` sofreu alteração?

2. **(Parte 2)** Explique, com base no `nm` e no `od`, por que a posição correspondente ao SysTick passou a apontar para a nova função sem qualquer alteração em `startup.c`. O que ocorreria se o `embedded-systick.c` e um outro arquivo definissem, ambos, uma função `systick_handler` comum?
3. **(Parte 3)** Apresente a tabela de previsões elaborada, bem como a tabela corrigida, e responda aos cinco pontos listados ao final da Parte 3.
4. **(Parte 4)** Compare o laço de cópia da `.data` sob `-O0` e sob `-O1`: quantas instruções possui cada versão, e o que desapareceu entre uma e outra? Explique por que a versão `-O2` não se liga com `-nostdlib`, e o que a opção `-fno-tree-loop-distribute-patterns` modificou. Cite a seção do manual do GCC que trata do assunto. Qual instrução efetua a chamada ao `main` em cada nível, e por que motivo ela se altera no `-O2` mediante essa opção?
5. **(Parte 5)** Para cada um dos três experimentos, explique a previsão formulada, o resultado obtido e a razão pela qual o programa não funcionaria na placa.
6. **(Parte 6)** Apresente o bloco `MEMORY` do `reserva.ld` e a saída de `--print-memory-usage`. Explique o erro de ligação relacionado a `g_table` e apresente a solução encontrada para que o `linker` descarte a tabela não utilizada.
7. **(Parte 7)** Responda às três perguntas relativas ao `embedded.map` e à diferença entre variável e símbolo.

! Entrega e Controle de Versão

A entrega consiste no envio das respostas do Relatório — redigidas em um arquivo `LEIAME.md` dentro da pasta `lab-03` — e da versão final dos arquivos-fonte da atividade (`startup.c`, `stm32f411-rom.ld` modificado e `embedded-systick.c`) para o seu repositório `seml-entregas`.

Para concluir a entrega e consolidar a filosofia de controle de versão:

1. Verifique as alterações antes do commit: execute `git status`. Observe os arquivos não rastreados. Diferentemente da semana anterior, desta vez geramos artefatos específicos do ARM (`.elf`, `.bin`, `.map`). Se você copiou o `.gitignore` sugerido na primeira semana, o Git já deve ignorar esses arquivos silenciosamente. Execute também:

```
foo@bar$ git diff
```

Isso mostrará linha por linha exatamente o que você alterou no *linker script* e no `startup.c`. Revisar o próprio *diff* antes de “empacotar” o commit é um dos hábitos mais valiosos do desenvolvimento de software.

2. Prepare e registre:

```
foo@bar$ git add .  
foo@bar$ git commit -m "Refatora tabela de vetores e resolve questoes do  
lab-03"
```

A sua mensagem deve descrever claramente a alteração funcional que foi feita no código (a inclusão de todas as interrupções do STM32F411, por exemplo).

3. Revise a história:

```
foo@bar$ git log --oneline
```

O seu repositório agora deve exibir uma linha do tempo clara: o commit inicial de configuração, o envio da compilação nativa (lab-02) e o commit atual. Você acaba de criar um registro reprodutível e histórico do seu progresso na disciplina.

4. Envie: `git push origin master`.

Referências

[1] RM0383 — STM32F411xC/E Reference Manual — STMicroelectronics. *Table 37 (Vector table for STM32F411xC/E)*.

[2] Using the GNU Compiler Collection (GCC) — seções *Common Function Attributes* (`weak`, `alias`), *Optimize Options* (`-O1`, `-O2`, `-ffunction-sections`, `-fno-tree-loop-distribute-patterns`) e *Standards* (as funções que um programa sem biblioteca C precisa fornecer).

[3] The GNU Linker (ld) — seções *Linker Scripts* (`KEEP`, `AT>`) e *Options* (`--gc-sections`, `--print-memory-usage`).